



Guia 5– Implementação do servidor BLE

Objetivos:

- Introdução ao ambiente de desenvolvimento do arduino
- Interrupções por hardware e software
- Leitura de sensores de temperatura, humidade e de distância.

Preparação do shield BLE

1. Prepare o shield de comunicações de acordo com as ligações ilustradas na Figura 1 e instale-o no arduino de acordo com a Figura 2. Para mais informações acerca das opções de parametrização do modo de funcionamento do shield consulte <https://www.botonroll.com/pt/shield-comunicacoes/2660-shield-bluetooth-ble-low-energy-p-arduino-itead.html>.

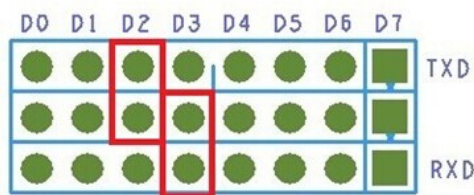


Figura 1 - Colocação dos jumpers



Figura 2 - Colocação do shield no arduino

2. Prepare o shield de comunicações de acordo com as ligações ilustradas na Figura 1 e instale-o no arduino de acordo com a Figura 2. Para mais informações acerca das opções de parametrização do modo de funcionamento do shield consulte

```
#include <SoftwareSerial.h>
#define HM11_RST 13

SoftwareSerial ble(2, 3);
int inByte;

void setup() {
  // rst HM-11 at power up
  pinMode(HM11_RST, OUTPUT);
  digitalWrite(HM11_RST, LOW);
  delay(100);
  digitalWrite(HM11_RST, HIGH);

  // initialize both serial ports:
  Serial.begin(9600);
  ble.begin(9600);
}

void loop() {
```

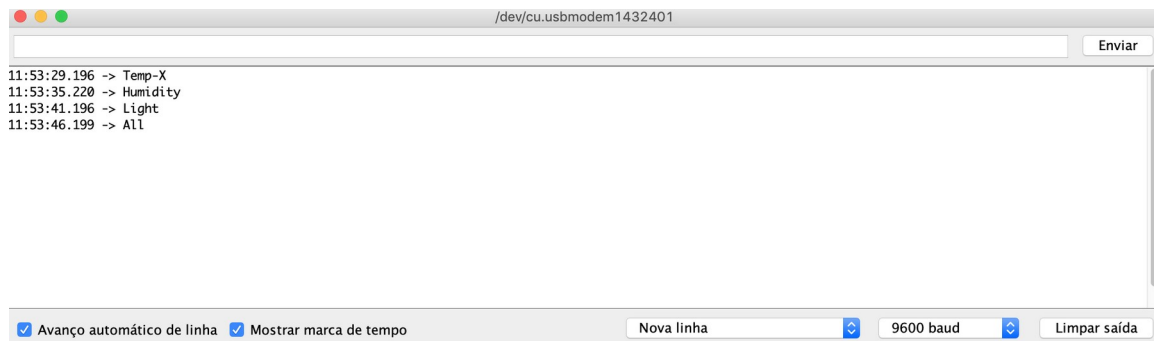


```
int delayScale;
if (ble.available()) {
  inByte = ble.read();
}
if (Serial.available()) {
  inByte = Serial.read();
}

if(inByte == 't'){
  ble.write("Temp-X\n");
  delayScale = 2;
  Serial.write("Temp-X\n");
}
if(inByte == 'h'){
  ble.write("Humidity\n");
  delayScale = 2;
  Serial.write("Humidity\n");
}
if(inByte == 'l'){
  ble.write("Light\n");
  delayScale = 1;
  Serial.write("Light\n");
}
if(inByte == 'X'){
  ble.write("All\n");
  delayScale = 1;
  Serial.write("All\n");
}

delay (delayScale*1000);
}
```

3. Faça o upload do sketch para o arduino e teste-o através da ferramenta de serial monitor do IDE.



Preparação da gateway

4. Procure a o Utilizando a aplicação hcitool procure o dispositivo BLE.

```
root@saffron:/home/pasg# hcitool dev
Devices:
    hci1    00:1A:7D:DA:71:15
    hci0    58:B0:35:92:B9:45
root@saffron:/home/pasg# hcitool lescan
LE Scan ...
3F:A4:AB:35:9F:96 (unknown)
7C:EC:79:DB:72:48 (unknown)
```



7C:EC:79:DB:72:48 C3PO

5. Ligue-se e verifique a recepção da informação.

```
root@saffron:/home/pasg# gatttool -b 7C:EC:79:DB:72:48 -I
[7C:EC:79:DB:72:48][LE]> CONNECT
Attempting to connect to 7C:EC:79:DB:72:48
Connection successful
Notification handle = 0x0012 value: 54 65 6d 70 2d 58 0a
Notification handle = 0x0012 value: 48 75 6d 69 64 69 74 79 0a
Notification handle = 0x0012 value: 4c 69 67 68 74 0a
Notification handle = 0x0012 value: 41 6c 6c 0a
[7C:EC:79:DB:72:48][LE]>
```

Criação da aplicação usando python

6. Efectue o download a biblioteca BLEserialbridge de https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=1&cad=rja&uact=8&ved=2ahUKewjN46OBsIrmAhWLyYUKHYNoCCYQFjAAegQIAhAB&url=https%3A%2F%2Fgithub.com%2Fakrv%2FBLEserialbridge&usg=AOvVaw1gYO95YyRd_tgzEZhExxE4

e leia as instruções presentes no ficheiro README.md, bem como no URL de um blog (<https://blog.akrv.xyz/?p=1003>) aí identificado.

7. Prepare o seu rasperian para utilizar a biblioteca

```
# sudo apt-get install build-essential python-pip3 git
# git clone https://github.com/akrv/BLEserialbridge
# cd BLEserialbridge
# pip3 install -r requirements.txt
```

8. Criei um novo programa python (arduinoBLE.py) que envia pedidos de uma forma rotativa e que imprime os dados das notificações.

```
import asyncio
from bleak import BleakClient

# Endereço MAC do módulo sensor
ADDRESS = "20:91:48:4D:D2:6B"
# UUID do serviço e da característica do HM-11
SERVICE_UUID = "0000ffe0-0000-1000-8000-00805f9b34fb"
CHAR_UUID = "00002901-0000-1000-8000-00805f9b34fb"

def notification_handler(sender, data: bytearray):
    """Função chamada sempre que há uma notificação BLE."""
    print("Aqui vai ter que ser Broker Client")
    try:
        print(data.decode("utf-8"))
    except UnicodeDecodeError:
        print(data)

async def main():
    async with BleakClient(ADDRESS) as client:
        if not client.is_connected:
            print("❌ Falha na ligação ao HM-11.")
        return
```



```
print("✅ Ligado ao HM-11!")

# Ativa notificações
await client.start_notify(CHAR_UUID, notification_handler)
print("notificações ativas.")

# Envia a primeira mensagem "h"
await client.write_gatt_char(CHAR_UUID, b"h")
print("Enviado: h")

index = 0
try:
    while True:
        # envia comandos alternadamente, como no original
        if index % 10 == 0:
            msg = b"l"
        elif index % 10 == 1:
            msg = b"t"
        elif index % 10 == 2:
            msg = b"h"
        elif index % 10 == 3:
            msg = b"x"
        else:
            msg = None

        if msg:
            await client.write_gatt_char(CHAR_UUID, msg)
            print(f"Enviado: {msg.decode()}")

        await asyncio.sleep(1)
        index += 1

except KeyboardInterrupt:
    print("\nInterrompido pelo utilizador.")

# Desativa notificações e desliga
await client.stop_notify(CHAR_UUID)
print("Notificações desativadas.")
print("Ligação terminada.")

if __name__ == "__main__":
    asyncio.run(main())
```

9. Para correr os programas em python precisam de invocar o processo python com o nome do ficheiro do programa e.g:

```
python3 arduinoBLE.py
```

Integração de tudo

10. Ligue o sensor DTH11 a um pino digital do arduino e adapte o seu sketch de forma que o arduino possa ler e imprimir os dados do sensor.

11. Volte ao sketch e adapte o código de envio para que transmita a informação sensorizada para a gateway.

12. Adapte também o programa de python de forma que possa ler e apresentar os dados da temperatura e da humidade.